# `na_values` and missing values in `read_csv`

**Status:** extends Practical 6 **Where it is used:** `backend/analytics.py` (`MISSING_MARKERS`, `read_dataset`, `missing_report`, `profile`), `scripts/profile_dataset.py` (`MISSING_MARKERS`, `read_dataset`, `profile`) **Closest lab concept:** Practical 6, `read_csv` and `df.shape` — `na_values` is one extra keyword on the same call, and it changes the *dtype* of every column it touches

---

## 1. What it is

By default `pd.read_csv` converts **empty cells** to `NaN` and nothing else. A truly blank field between two commas becomes missing. A field containing the text `?` is a perfectly good string, so pandas reads it as the string `?` and reports the column as complete.

`na_values` changes that rule. It is a list of strings, and every cell whose text matches one of them is converted to `NaN` instead. This is the shape the project uses, with its constant filled in — the real call is quoted in full in section 2:

```
    df = pd.read_csv(path, na_values=MISSING_MARKERS, keep_default_na=True)
```

where `MISSING_MARKERS` is `["?", "-9", "NA", "N/A", ""]`.

Practical 6 used `read_csv` without this keyword, on lab CSVs where missing really was a blank cell. Nothing was missed, because blank was the only convention in play. Real datasets do not all agree on one convention, and telling pandas about the extra ones is the entire job of this keyword.

Two things change downstream as a result, and both matter:

1. `df.isnull()` now counts those cells.
2. The column's dtype is decided *after* the conversion. A column of numbers with some `?` in it becomes `float64` (because `NaN` needs a float) instead of `object`. A column of numbers with no markers in it stays `int64`.

That second effect is the one that broke the project.

---

## 2. Why the project needed it

The UCI files use their own conventions, documented in the `.names` files and in the comment at `analytics.py:24-26`:

```
# UCI files mark unknown values with ? and not-applicable with -9. Without
# these markers pandas reads the whole column as text and every numeric
# count comes back zero.
MISSING_MARKERS = ["?", "-9", "NA", "N/A", ""]
```

So the project's answer to "what counts as missing here?" is a module constant, defined once and passed to every read. `analytics.py:62-69`, quoted in full:

```
def read_dataset(name):
    """Read one dataset with missing markers converted to NaN."""
    path = os.path.join(DATA_DIR, name)

    if not os.path.exists(path):
        raise FileNotFoundError(f"dataset not found: {name}")

    return pd.read_csv(path, na_values=MISSING_MARKERS, keep_default_na=True)
```

`scripts/profile_dataset.py:22-30` has its own copy of the same constant and the same call, which is deliberate: the profiler is a standalone script that scores files *before* they are admitted, so it cannot

import from the module it is judging. Two copies, one source of truth in review, and `profile_dataset.py:23-28` carries the explanation so a reader who finds one file can follow the reasoning to the other.

The consequence of getting this wrong is not a wrong count. It is that `select_dtypes(include="number")` finds nothing, and every analytic in the project silently returns empty — see `docs/concepts/pandas-select-dtypes.md`, section 5.

---

## 3. The four pieces actually used

### 3.1 `na_values=MISSING_MARKERS`

`MISSING_MARKERS` is a plain list of strings, passed straight through. Note what is **not** in it: `-9` is the string `"-9"`, not the integer `-9`. `na_values` matches against cell *text*, so the list holds strings throughout. An integer `-9` in the list would silently match nothing, and the bug in section 5 would look like this one.

The `""` at the end is redundant — pandas already treats empty cells as missing — but it is listed explicitly so that reading the constant answers "what counts as missing in this project" without needing to know pandas' defaults. `keep_default_na=True` (next section) is what actually guarantees the blank behaviour.

### 3.2 `keep_default_na=True`

This is the other half of the rule, and it is the default, so passing it explicitly changes nothing today. It is there on purpose: it states that pandas' own default list (empty cells, `NA`, `N/A`, `NULL`, `NaN`, `-`, and friends) stays switched on *in addition to* `MISSING_MARKERS`.

The distinction matters because `na_values` alone does **not** replace the defaults — it adds to them. With `keep_default_na=False` you would be saying "only these five strings are missing", and every other convention would come back as live text. Since the project's four datasets come from different archives with different habits, both settings on is the honest position: union, not replace.

### 3.3 `df.isnull().sum()` — per column, then filter

`analytics.missing_report`, `analytics.py:250-254`:

```
def missing_report(df):
    """Return missing value counts per column."""
    counts = df.isnull().sum()

    return to_native({str(k): int(v) for k, v in counts.items() if v > 0})
```

`df.isnull()` is the Practical 6 boolean-filtering shape — it returns a frame of `True`/`False`, one per cell — but applied to the whole frame rather than to a single comparison. `.sum()` on a boolean frame counts `True` as 1 per column, so the result is a Series with one count per column, which is exactly what `df.shape[1]` indexed by column name expects.

The `if v > 0` in the comprehension is the part worth copying. A column with zero missing is not interesting and the API should not carry it, so clean columns are dropped rather than reported as `0`. Against `heart_disease.csv` this returns exactly:

```
{'ca': 4, 'thal': 2}
```

Six missing cells in a 303x14 frame, in two columns, out of fourteen. That is the honest answer, and section 5 is about the version that was not.

### 3.4 `.sum().sum()` — cells, not columns

`profile_dataset.py:52-53`:

```
    total_cells = df.shape[0] * df.shape[1]
    missing_cells = int(df.isnull().sum().sum())
```

The first `.sum()` collapses to one count per column. The second collapses the columns to a single integer. Both are `.sum()` on the same kind of object, so the second line is just the first applied again — a Series of counts, summed.

`df.shape` is doing the other half of the job here, exactly as in Practical 6: `shape[0]` is rows, `shape[1]` is columns, and `rows * columns` is the number of cells the missing ones are counted against. `profile_dataset.py:66` turns the pair into the scorecard number:

```
        "missing_pct": round(100 * missing_cells / total_cells, 2),
```

This is the figure spec section 4 gates datasets on, `MAX_MISSING_RATIO = 0.20`.

---

## 4. Real bug one: the Parkinsons doubled header

**This project got it wrong first.** `scripts/stage_datasets.py` exists to attach real column names to the UCI downloads that ship without one. Most do not have a header. `parkinsons.csv` does.

`stage_parkinsons` therefore reads the first line, renames it, and writes the rest — `stage_datasets.py:103-105`:

```
    original_header = lines[0].split(",")
    columns = [PARKINSONS_RENAME.get(c, c) for c in original_header]
    write_with_header("parkinsons.csv", columns, lines[1:])
```

`lines[1:]` is the fix. The version that shipped first passed the whole `lines` list, so the file's own header row was written back out as if it were data — one header on top of a header that was already there.

Nothing crashed. `pd.read_csv` read the doubled file perfectly happily: it took row one as the column names and treated row two as the first record. The file had **196** rows instead of 195.

`scripts/profile_dataset.py` is what caught it, because the profiler was written specifically to print per-dataset diagnostics rather than trust a file. Reproducing the doubled-header read on this project's real `raw/parkinsons.csv`:

```
shape (196, 24)
numeric cols 0
status col values {'1': 147, '0': 48, 'status': 1}
status dtype str
target found? ['status']
```

Three symptoms, all of them real:

1. **196 rows, not 195.** The extra row is the header that was treated as a record.
2. **`status` contains the literal string `"status"`.** The renamed header is sitting in the data. This is the giveaway — a clinical status column has two values, 0 and 1, and never the word "status".
3. **Zero numeric columns.** Every value in the file is now text because every row is shifted one column left relative to its name, so `select_dtypes(include="number")` matched nothing and every numeric statistic came back empty.

Why the third symptom matters more than the first two: it is the one that reaches the API. An off-by-one row count is a number a human might glance past. Zero numeric columns turns `describe_numeric` and `group_means` into `{}` and `[]` through the silent-empty path in `select_dtypes`, which is indistinguishable from a genuinely uninteresting dataset. That is exactly the failure mode `docs/concepts/pandas-select-dtypes.md` section 5 describes.

The detection rule is worth stating on its own, because it generalises: **if a categorical column has a value that is also a column name, you have read a header as data.**

---

## 5. Real bug two: heart_disease reported 0% missing

The second bug is quieter and worse, because it produced a confident wrong number rather than an empty one.

Before `na_values` was added, `pd.read_csv` was called bare. The `?` values in the Cleveland heart data are not blank cells, so pandas treated each one as an ordinary string. Measured on the current file:

```
heart_disease.csv naive numeric: 12 naive missing_cells: 0 raw pct 0.0
```

So `isnull()` found nothing, `missing_pct` came back `0.0`, and the report said heart data was complete. It is not — `ca` (number of major vessels coloured by fluoroscopy) and `thal` (thalassemia test) both have genuinely unmeasured entries.

The two errors compounded, which is why this one was worth a fix rather than a note. The `?` strings made those two columns `object` dtype, which dropped the numeric count from 13 to 12 — so the dataset *also* looked like it had fewer measurements than it should. Fixing `na_values` fixed both: `ca` and `thal` become `float64`, the numeric count returns to 13, and the missing report becomes truthful.

Current, correct output for the same file:

```
heart shape (303, 14)
missing_report {'ca': 4, 'thal': 2}
missing_pct 0.14
numeric ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope'
```

The lesson is narrow and transferable: **a missing-value count of exactly zero is a claim, not a result.** The profiler printed the percentage for every dataset specifically so that a suspicious zero would show up as a suspicious zero, next to datasets with realistic rates like mammographic masses at 16.69%. A metric that can only ever look good deserves more suspicion than one that looks bad.

---

## 6. Honest limitation: blank is not zero, and `na_values` cannot help

Here is a problem this project did **not** solve, and does not claim to.

`pima_diabetes.csv` reports **0.0% missing**. That is not a bug in the reading — it is a fact about the dataset. The Pima Indians diabetes file does not leave missing cells blank and does not write `?`. It encodes unknown values as a literal `0`, because `0` already sits in that column's range as a legitimate measurement and the file's authors had no other way to say "we did not measure this".

So a Glucose of 0 means one of two things: the person genuinely had no glucose measurement taken, or their glucose was somehow zero. The second is not physiologically plausible. Neither is a BMI of 0, nor an Insulin of 0, nor a SkinThickness of 0. Counting the zeros in this project's staged copy:

```
{'Pregnancies': 111, 'Glucose': 5, 'BloodPressure': 35, 'SkinThickness': 227,
 'Insulin': 374, 'BMI': 11, 'DiabetesPedigreeFunction': 0, 'Age': 0, 'Outcome': 500}
```

374 of 768 Insulin readings and 227 of 768 SkinThickness readings are zero. Almost none of those are real measurements. Yet `df.isnull()` reports zero missing cells, because pandas did exactly what it was told: there are no blank cells in this file.

**`na_values` cannot catch this, and nothing in `read_csv` can.** It works by matching cell *text* against a list you supply. The text here is `"0"`, which is also a valid measurement. The only way to catch it would be a per-column rule that says "in `Insulin`, the value 0 means missing" — and a blanket rule of that shape would destroy the two columns where 0 *is* legitimate: `Pregnancies` (111 zeros are real) and `Outcome` (500 zeros are the negative class). This project does not apply that rule, because a rule that is wrong for 611 real measurements is worse than a known limitation that is documented.

The limitation is not hypothetical here, and it has a visible effect. The zeros drag `BMI`'s statistics downward, and they are sitting inside the numbers the risk bands are computed from.

`analytics.py:31-42` is the honest handling, and the reasoning is written down in the source:

```
# Risk band cut points per dataset column. Modelled on the BMI
# categorisation problem from Practical 1, same if/elif shape.
#
# Cut points are the dataset's own quartiles rather than textbook clinical
# thresholds, because these are cohort risk bands, not diagnosis. Using
# textbook cutoffs made the bands useless: BMI quartiles are 27.3 / 32 /
# 36.6, so standard cutoffs of 18.5 / 25 / 30 put 472 of 768 patients in
# the top band. Quartiles give roughly equal groups.
#
# Glucose is the exception and keeps clinical cutoffs of 100 and 140,
# because they align with the diabetes screening threshold and produce a
# clean monotonic outcome rate: 8%, 29%, 50%, 82%.
```

`RISK_THRESHOLDS` therefore uses `[27.3, 32.0, 36.6]` for BMI — which are this dataset's own quartile boundaries, confirmed by `describe()`:

```
25%    27.300000
50%    32.000000
75%    36.600000
```

Quartiles adapt to whatever the column actually contains, including the inflated low tail from those 11 impossible BMI zeros. Clinical cutoffs would not: they would place 472 of 768 patients in the top band, which is the "made the bands useless" outcome in the comment.

Two things to be clear about, because this is easy to misread:

- **Quartiles are not a fix.** They do not repair the data. They pick cut points that survive the damage, so the band chart still communicates something true about the cohort.
- **Glucose keeps clinical cutoffs on purpose.** The comment records the reason: the outcome rate rises monotonically across the bands (8%, 29%, 50%, 82%), so the cut points carry real information rather than just equalising group sizes.

Anyone extending this should know that `missing_pct` in `profile_report.csv` means "cells pandas considers blank", which is a lower bound on the truth, not the truth.

---

## 7. Questions this should answer

- By default, which cells does `pd.read_csv` turn into `NaN`, and why does a cell containing `?` not qualify?
- Why does `MISSING_MARKERS` contain the **string** `"-9"` rather than the integer `-9`, and what would happen if it held the integer?
- What does `keep_default_na=True` do *in addition to* `na_values`, as opposed to replacing it?
- `df.isnull().sum()` gives one count per column. What does the second `.sum()` in `df.isnull().sum().sum()` do, and which of the two does `MAX_MISSING_RATIO` need?
- The Parkinsons file had a doubled header. What three symptoms did the profiler report, and why was "0 numeric columns" the most dangerous of them?
- Why is `na_values` unable to detect the missing values in `pima_diabetes.csv`, and what is the difference between the mitigation in `RISK_THRESHOLDS` and an actual fix?